

Лекция. Фоновая работа с WorkManager

API WorkManager для Android упрощает фоновую работу. WorkManager может создавать задачи, которые можно запрашивать, повторно использовать и связывать в цепочку. WorkManager является рекомендуемым планировщиком задач на Android. В этом руководстве вы узнаете, как пользоваться WorkManager: создавать рабочих, использовать запросы на выполнение работы и объединять рабочих в цепочки в соответствии с потребностями вашего приложения.

В этом руководстве рассказывается о **WorkManager**, совместимой с обратным развитием, гибкой и простой библиотеке для отложенной фоновой работы. WorkManager является рекомендуемым планировщиком задач на Android для отложенной работы с гарантией выполнения.

Необходимые условия

- Знание StateFlow и ViewModel. Если вы впервые знакомитесь с этими классами, ознакомьтесь с ViewModel и State в Compose Codelab (специально для ViewModel и State) или Read and update data with Room Codelab (специально для Flow и StateFlow).
- Знание репозитория и инъекции зависимостей. Чтобы получить более подробную информацию, ознакомьтесь с Add repository и Manual DI.
- Уметь реализовывать корутины в своем приложении.

Что вы узнаете

- Как добавить WorkManager в свой проект.
- Как запланировать простую задачу.
- Как настраивать входные и выходные параметры для рабочих.
- Как объединять рабочих в цепочку.

Что вы будете делать

- Измените стартовое приложение для использования WorkManager.
- Реализуйте рабочий запрос на размытие изображения.
- Реализуйте последовательную группу работ, соединив их в цепочку.
- Передавать данные в запланированную работу и из нее.

Что вам понадобится

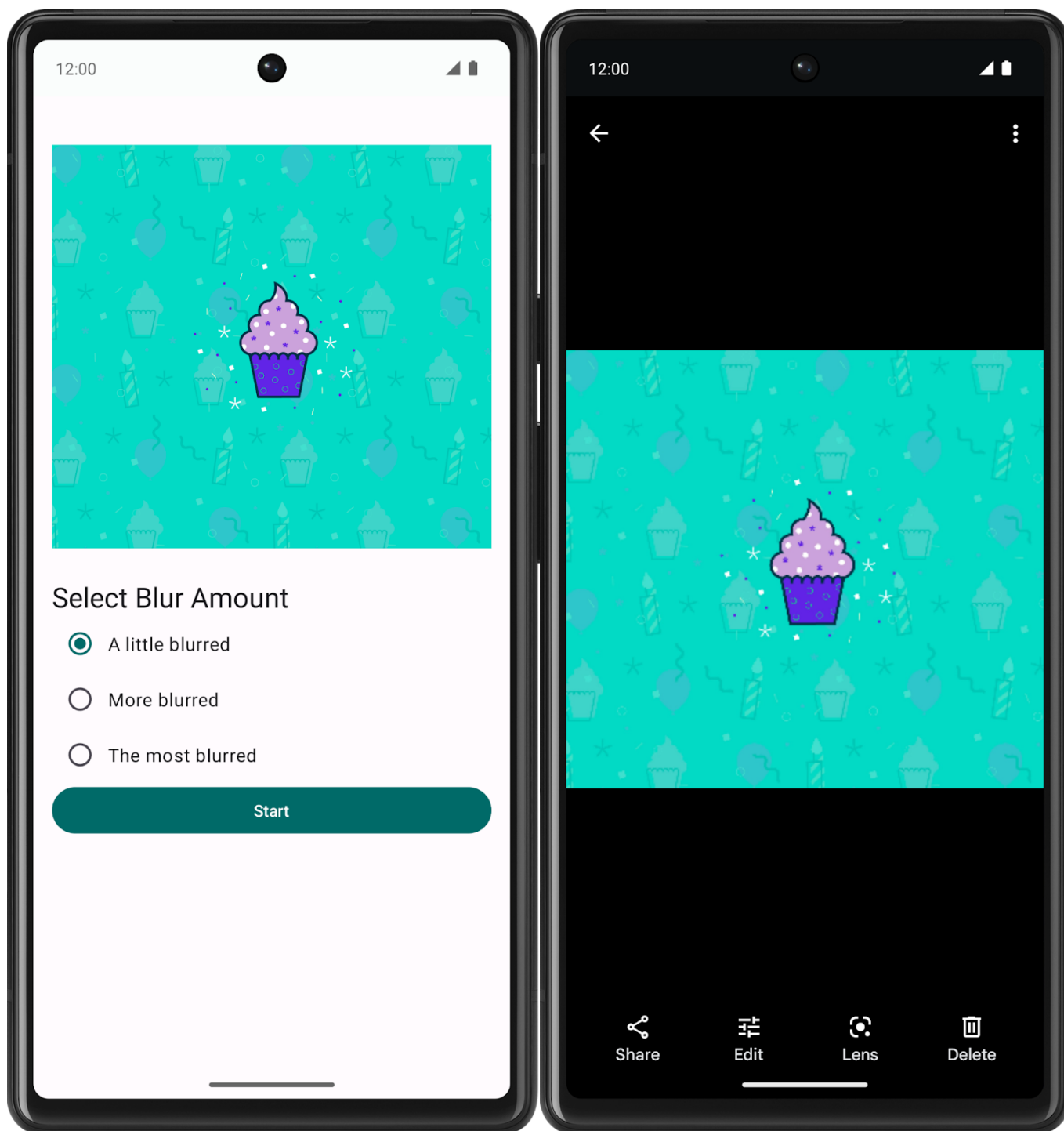
- Последняя стабильная версия Android Studio
- Подключение к интернету

Обзор приложения

В наши дни смартфоны почти слишком хорошо умеют делать снимки. Прошли те времена, когда фотограф мог сделать достоверно размытый снимок чего-то загадочного.

В этом коделабе вы будете работать над Blur-O-Matic, приложением, которое размывает фотографии и сохраняет результаты в файл. Что это было - Лохнесское чудовище или игрушечная подводная лодка? С

Blur-O-Matic никто никогда не узнает!



На экране есть радиокнопки, с помощью которых можно выбрать степень размытия изображения. Нажав кнопку «Старт», вы размываете и сохраняете изображение.

Сейчас приложение не применяет размытие и не сохраняет конечное изображение.

В этом руководстве мы рассмотрим добавление WorkManager в приложение, создание рабочих для очистки временных файлов, которые создаются при размытии изображения, размытие изображения и сохранение конечной копии изображения, которую можно просмотреть, нажав кнопку «Посмотреть файл». Вы также узнаете, как отслеживать состояние фоновой работы и соответствующим образом обновлять пользовательский интерфейс приложения.

3. Изучите стартовое приложение Blur-O-Matic

Получите стартовый код Чтобы начать работу, загрузите стартовый код:

file_downloadСкачать zip

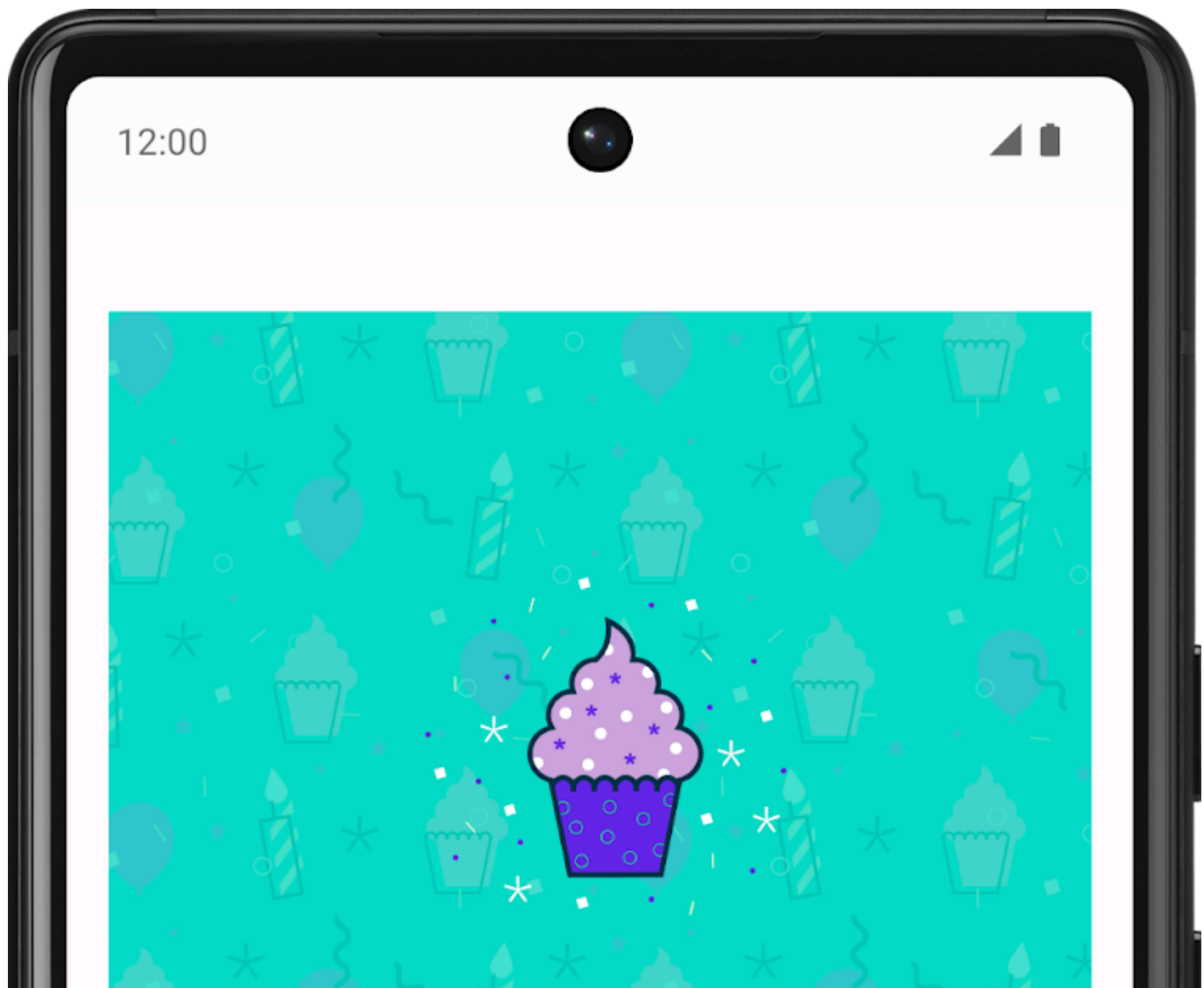
Также вы можете клонировать репозиторий GitHub для кода:

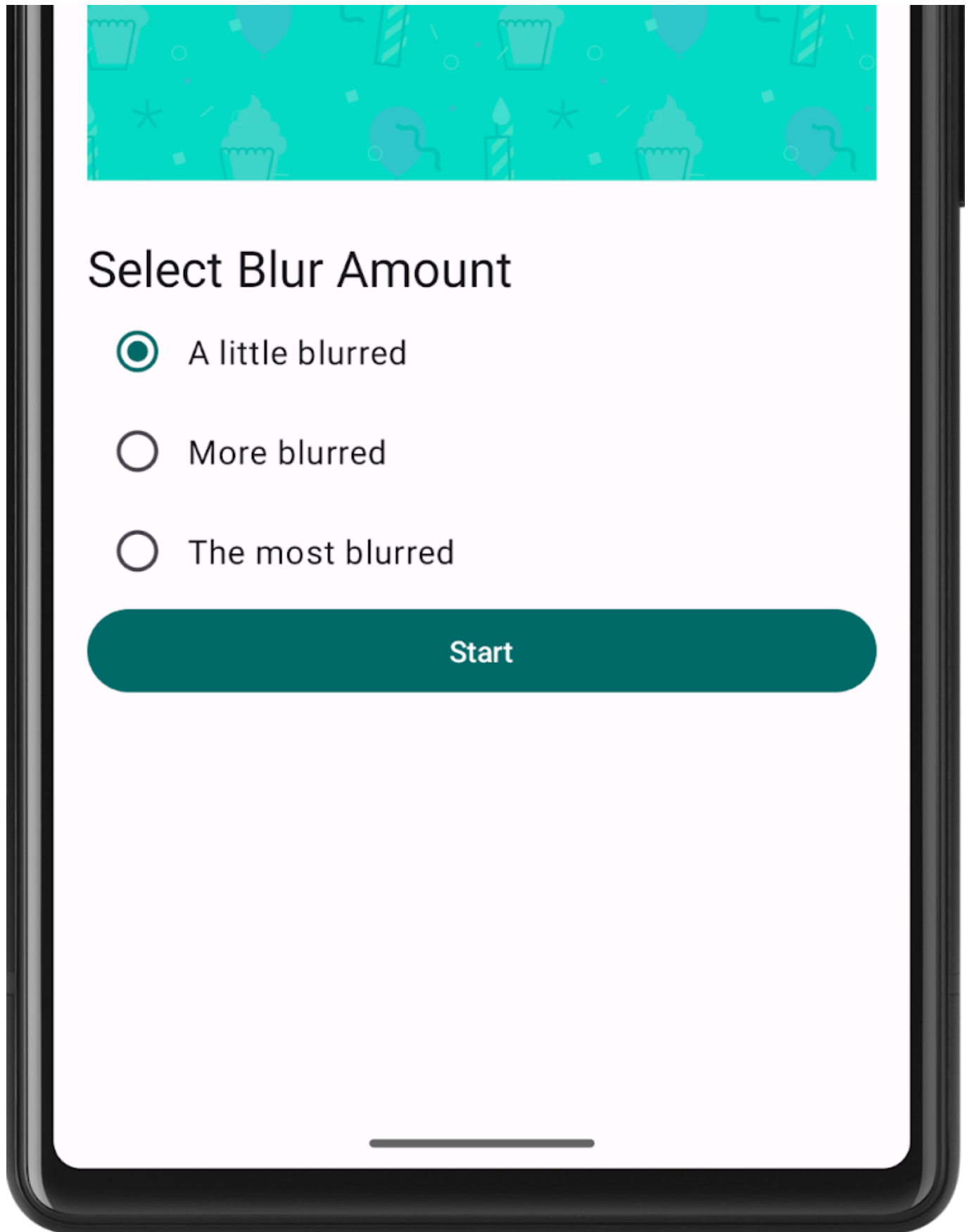
```
$ git clone https://github.com/google-developer-training/basic-android-kotlin-compose-training-workmanager.git
$ cd basic-android-kotlin-compose-training-workmanager
$ git checkout starter
```

Запустить стартовый код

Чтобы ознакомиться со стартовым кодом, выполните следующие шаги:

Откройте проект со стартовым кодом в Android Studio. Запустите приложение на Android-устройстве или в эмуляторе.





{style=«width:400px»}

На экране есть радиокнопки, позволяющие выбрать степень размытия изображения. Когда вы нажимаете кнопку «Старт», приложение размывает и сохраняет изображение.

В данный момент приложение не применяет никакого размытия при нажатии кнопки «Старт».

Прохождение начального кода В этом задании вы знакомитесь со структурой проекта. В следующих списках представлены важные файлы и папки проекта.

WorkerUtils: Удобные методы, которые вы позже используете для отображения уведомлений и код для сохранения растрового изображения в файл. **BlurViewModel:** Эта модель представления хранит состояние приложения и взаимодействует с репозиторием. **WorkManagerBluromaticRepository:** Класс, в котором начинается фоновая работа с WorkManager. **Константы:** Статический класс с некоторыми константами, которые вы используете во время коделаба. **BluromaticScreen:** Содержит композитные функции для пользовательского интерфейса и взаимодействует с моделью BlurViewModel. Составные функции показывают изображение и включают радиокнопки для выбора желаемого уровня размытия.

4. Что такое WorkManager?

WorkManager - это часть Android Jetpack и архитектурный компонент для фоновой работы, которая требует сочетания оппортунистического и гарантированного выполнения. Оппортунистическое выполнение означает, что WorkManager выполняет вашу фоновую работу, как только может. Гарантированное выполнение означает, что WorkManager позаботится о логике запуска вашей работы в различных ситуациях, даже если вы перейдете из приложения.

WorkManager - это невероятно гибкая библиотека, которая имеет множество дополнительных преимуществ. Вот некоторые из них:

Поддержка как асинхронных разовых, так и периодических задач. Поддержка ограничений, таких как состояние сети, объем памяти и состояние зарядки. Цепочка сложных рабочих запросов, например, параллельное выполнение работы. Выходные данные одного рабочего запроса используются в качестве входных данных для следующего. Совместимость с API-уровнем вплоть до API-уровня 14 (см. примечание). Работа с сервисами Google Play или без них. Соблюдение лучших практик по обеспечению работоспособности системы. Поддержка удобного отображения состояния рабочих запросов в пользовательском интерфейсе приложения.

Примечание: WorkManager работает поверх нескольких API, таких как JobScheduler и AlarmManager. WorkManager выбирает нужные API для использования в зависимости от условий, например уровня API устройства пользователя. Чтобы узнать больше, ознакомьтесь с документацией по WorkManager «Планирование заданий с помощью WorkManager» и WorkManager.

5. Когда использовать WorkManager

Библиотека WorkManager - отличный выбор для задач, которые вам необходимо выполнить. Выполнение этих задач не зависит от того, продолжает ли приложение работать после того, как работа поставлена в очередь. Задачи выполняются, даже если приложение закрыто или пользователь вернулся на главный экран.

Некоторые примеры задач, которые хорошо использовать в WorkManager:

Периодический запрос последних новостей. Применение фильтров к изображению с последующим его сохранением. Периодическая синхронизация локальных данных с сетью. WorkManager - это один из

вариантов выполнения задачи вне основного потока, но он не является универсальным средством для выполнения всех типов задач вне основного потока. Корутины - это еще один вариант, о котором говорилось в предыдущих кодолабораториях.

Подробнее о том, когда следует использовать WorkManager, читайте в руководстве по фоновой работе.

6. Добавьте WorkManager в свое приложение

Для WorkManager требуется следующая зависимость gradle. Она уже включена в файл сборки:

app/build.gradle.kts

```
dependencies {  
    // WorkManager dependency  
    implementation("androidx.work:work-runtime-ktx:2.8.1")  
}
```

Вы должны использовать в своем приложении самую актуальную стабильную версию work-runtime-ktx.

Если вы измените версию, не забудьте нажать Sync Now, чтобы синхронизировать проект с обновленными gradle-файлами.

7. Основы WorkManager

Есть несколько классов WorkManager, о которых вам нужно знать:

Worker / CoroutineWorker: Worker - это класс, который выполняет работу синхронно в фоновом потоке. Поскольку нас интересует асинхронная работа, мы можем использовать CoroutineWorker, который имеет взаимодействие с Kotlin Coroutines. В этом приложении вы расширяетесь за счет класса CoroutineWorker и переопределяете метод doWork(). В этот метод вы помещаете код для фактической работы, которую вы хотите выполнить в фоновом режиме. **WorkRequest:** Этот класс представляет собой запрос на выполнение некоторой работы. В WorkRequest вы определяете, должен ли рабочий выполняться один раз или периодически. На WorkRequest также можно наложить ограничения, которые требуют выполнения определенных условий перед выполнением работы. Например, устройство должно быть заряжено перед началом выполнения запрошенной работы. Вы передаете CoroutineWorker в процессе создания WorkRequest. **WorkManager:** Этот класс фактически планирует ваш WorkRequest и обеспечивает его выполнение. Он планирует выполнение WorkRequest таким образом, чтобы распределить нагрузку на системные ресурсы, соблюдая при этом указанные вами ограничения. В вашем случае вы определяете новый класс BlurWorker, который содержит код для размытия изображения. Когда вы нажимаете кнопку Start, WorkManager создает и затем регистрирует объект WorkRequest.

8. Создание BlurWorker

На этом шаге вы берете изображение в папке res/drawable под названием android_cupcake.png и запускаете на нем несколько функций в фоновом режиме. Эти функции размывают изображение.

Щелкните правой кнопкой мыши на пакете com.example.bluromatic.workers в панели проекта Android и выберите New -> Kotlin Class/File. Назовите новый Kotlin-класс BlurWorker. Расширьте его из CoroutineWorker, добавив необходимые параметры конструктора. workers/BlurWorker.kt

```
import androidx.work.CoroutineWorker
import androidx.work.WorkerParameters
import android.content.Context

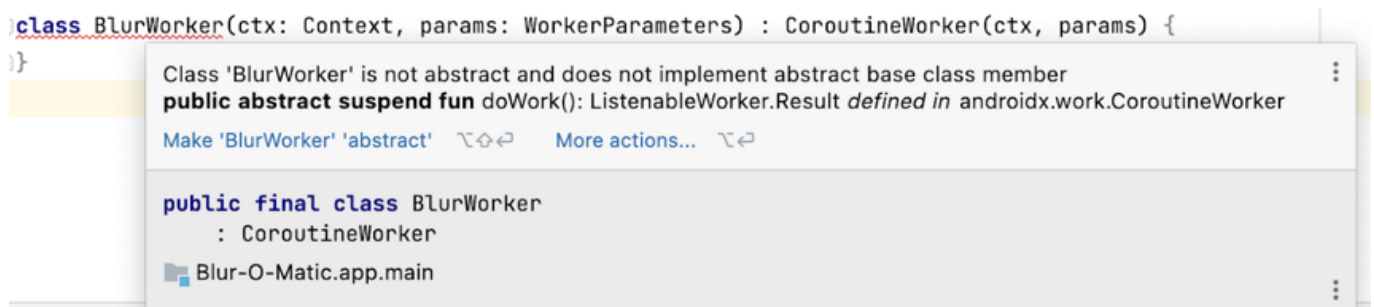
class BlurWorker(ctx: Context, params: WorkerParameters) : CoroutineWorker(ctx,
params) {
}
```

Класс BlurWorker расширяет класс CoroutineWorker вместо более общего класса Worker. Реализация функции doWork() класса CoroutineWorker является приостанавливающей функцией, что позволяет ему выполнять асинхронный код, который не может выполнить Worker. Как подробно описано в руководстве Threading in WorkManager, «CoroutineWorker - это рекомендуемая реализация для пользователей Kotlin».

В этот момент Android Studio рисует красную волнистую линию под классом BlurWorker, которая указывает на ошибку.

```
class BlurWorker(ctx: Context, params: WorkerParameters) : CoroutineWorker(ctx, params) {
}
```

Если навести курсор на текст class BlurWorker, IDE отобразит всплывающее окно с дополнительной информацией об ошибке.

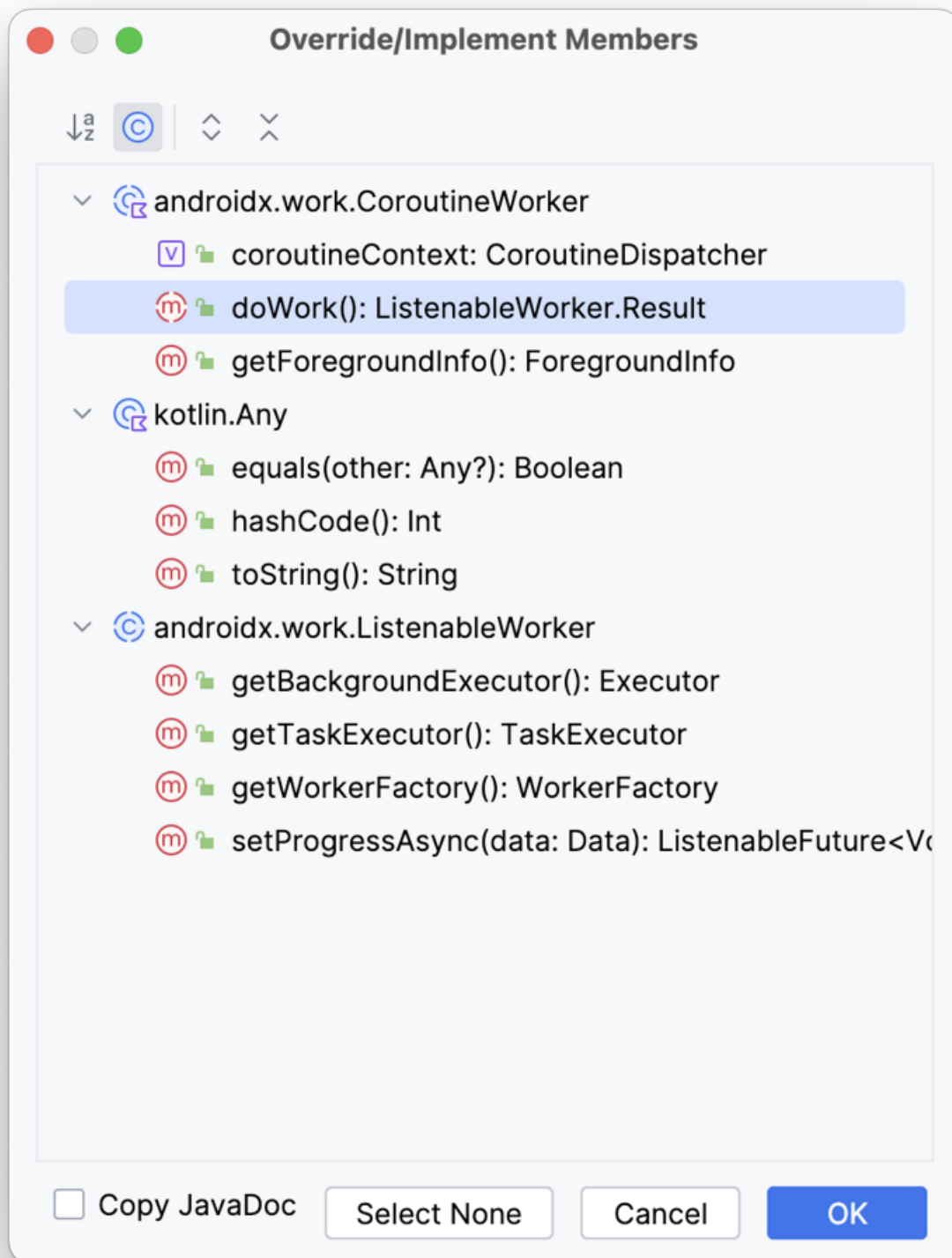


Сообщение об ошибке указывает на то, что вы не переопределили метод doWork(), как это требуется.

В методе doWork() напишите код, чтобы размыть показанное изображение кекса.

Выполните следующие шаги, чтобы исправить ошибку и реализовать метод doWork():

Установите курсор внутри кода класса, щелкнув по тексту «BlurWorker». В меню Android Studio выберите Code > Override Methods... Во всплывающем окне Override Members выберите doWork(). Нажмите ОК.



{style=«width:400px»}

Непосредственно перед объявлением класса создайте переменную с именем TAG и присвойте ей значение BlurWorker. Обратите внимание, что эта переменная не связана конкретно с методом doWork(), но вы используете ее позже в вызовах Log().

workers/BlurWorker.kt


```
private const val TAG = «BlurWorker»

class BlurWorker(ctx: Context, params: WorkerParameters) : CoroutineWorker(ctx,
params) {
    ...
}
```

Чтобы лучше видеть, когда выполняется работа, вам нужно использовать функцию `WorkerUtil.makeStatusNotification()`. Эта функция позволяет легко вывести баннер с уведомлением в верхней части экрана. Внутри метода `doWork()` используйте функцию `makeStatusNotification()`, чтобы вывести уведомление о состоянии и сообщить пользователю, что рабочий процесс размытия запущен и размывает изображение.

workers/BlurWorker.kt

```
import com.example.bluromatic.R
...
override suspend fun doWork(): Result {

    makeStatusNotification(
        applicationContext.resources.getString(R.string.blurring_image),
        applicationContext
    )
    ...
}
```

Добавьте блок кода `try...catch` с возвратом, в котором выполняется фактическая работа с размытием изображения. workers/BlurWorker.kt

```
...
    makeStatusNotification(
        applicationContext.resources.getString(R.string.blurring_image),
        applicationContext
    )

    return try {
    } catch (throwable: Throwable) {
    }
    ...
}
```

В блоке `try` добавьте вызов `Result.success()`. В блоке `catch` добавьте вызов `Result.failure()`.

Примечание: `WorkManager` использует `Result.success()` и `Result.failure()` для указания окончательного статуса выполняемого рабочего запроса.

workers/BlurWorker.kt

```
...
        makeStatusNotification(
            applicationContext.resources.getString(R.string.blurring_image),
            applicationContext
        )

        return try {
            Result.success()
        } catch (throwable: Throwable) {
            Result.failure()
        }
    }
...

```

В блоке try создайте новую переменную с именем `picture` и заполните ее растровым изображением, полученным в результате вызова метода `BitmapFactory.decodeResource()` и передачи пакета ресурсов приложения и идентификатора ресурса изображения кекса.

workers/BlurWorker.kt

```
...
        return try {
            val picture = BitmapFactory.decodeResource(
                applicationContext.resources,
                R.drawable.android_cupcake
            )

            Result.success()
        }
    }
...

```

Размойте растр, вызвав функцию `blurBitmap()` и передав в нее переменную `picture` и значение 1 (один) для параметра `blurLevel`. Сохраните результат в новой переменной с именем `output`.

Примечание: В последующих инструкциях вы будете передавать переменную для параметра `blurLevel`.

workers/BlurWorker.kt

```
...
            val picture = BitmapFactory.decodeResource(
                applicationContext.resources,
                R.drawable.android_cupcake
            )

            val output = blurBitmap(picture, 1)

            Result.success()
        }
    }
...

```

Создайте новую переменную `outputUri` и заполните ее вызовом функции `writeBitmapToFile()`. В вызове функции `writeBitmapToFile()` передайте в качестве аргументов контекст приложения и переменную `output`.

workers/BlurWorker.kt

```
...  
        val output = blurBitmap(picture, 1)  
  
        // Write bitmap to a temp file  
        val outputUri = writeBitmapToFile(applicationContext, output)  
  
        Result.success()  
...
```

Добавьте код для отображения уведомления пользователю, содержащего переменную `outputUri`.

workers/BlurWorker.kt

```
...  
        val outputUri = writeBitmapToFile(applicationContext, output)  
  
        makeStatusNotification(  
            «Выходные данные - $outputUri»,  
            applicationContext  
        )  
  
        Result.success()  
...
```

В блоке `catch` запишите сообщение об ошибке, указывающее на то, что при попытке размыть изображение произошла ошибка. В вызов `Log.e()` передается ранее определенная переменная `TAG`, соответствующее сообщение и возникающее исключение.

workers/BlurWorker.kt

```
...  
        } catch (throwable: Throwable) {  
            Log.e(  
                TAG,  
  
                applicationContext.resources.getString(R.string.error_applying_blur),  
                throwable  
            )  
            Result.failure()  
        }  
...
```

CoroutineWorker по умолчанию запускается как Dispatchers.Default, но его можно изменить, вызвав withContext() и передав нужный диспетчер.

Создайте блок withContext(). Внутри вызова withContext() передайте Dispatchers.IO, чтобы лямбда-функция выполнялась в специальном пуле потоков для потенциально блокирующих операций ввода-вывода. Переместите ранее написанный код return try...catch в этот блок.

```
...
    return withContext(Dispatchers.IO) {
        return try {
            // ...
        } catch (throwable: Throwable) {
            // ...
        }
    }
...
```

Android Studio выдает следующую ошибку, поскольку нельзя вызвать return из лямбда-функции.

```
return withContext(Dispatchers.IO) { this: CoroutineScope
```

```
return try {
```

'return' is not allowed here

Change to 'return@withContext' ↵ ↵ ↵ More actions... ↵ ↵

```
)
```

Эту ошибку можно исправить, добавив метку, как показано во всплывающем окне.

```
...
    //return try {
    return@withContext try {
...

```

Поскольку этот Worker выполняется очень быстро, рекомендуется добавить в код задержку, чтобы эмулировать более медленную работу.

Внутри лямбды withContext() добавьте вызов служебной функции delay() и передайте константу DELAY_TIME_MILLIS. Этот вызов предназначен исключительно для коделаба, чтобы обеспечить задержку между сообщениями уведомлений.

```
import com.example.bluromatic.DELAY_TIME_MILLIS
import kotlinx.coroutines.delay

...

return withContext(Dispatchers.IO) {

    // This is an utility function added to emulate slower work.
    delay(DELAY_TIME_MILLIS)

    val picture = BitmapFactory.decodeResource(
...

```

9. Обновление репозитория WorkManagerBluromaticRepository

Репозиторий управляет всеми взаимодействиями с WorkManager. Такая структура соответствует принципу разделения забот и является рекомендуемым архитектурным паттерном Android.

В файле `data/WorkManagerBluromaticRepository.kt`, внутри класса `WorkManagerBluromaticRepository`, создайте приватную переменную `workManager` и сохраните в ней экземпляр `WorkManager`, вызвав `WorkManager.getInstance(context)`.

`data/WorkManagerBluromaticRepository.kt`

```
import androidx.work.WorkManager
...
class WorkManagerBluromaticRepository(context: Context) : BluromaticRepository {

    // Новый код
    private val workManager = WorkManager.getInstance(context)
...

```

Создание и регистрация запроса на выполнение работы в WorkManager Итак, пора создать `WorkRequest` и указать `WorkManager` на его выполнение! Существует два типа рабочих запросов:

`OneTimeWorkRequest`: Рабочий запрос, который выполняется только один раз. Периодический запрос (`PeriodicWorkRequest`): Рабочий запрос, который выполняется несколько раз в течение цикла. Вы хотите, чтобы изображение размывалось только один раз при нажатии кнопки «Пуск».

Эта работа выполняется в методе `applyBlur()`, который вызывается при нажатии кнопки «Пуск».

Следующие шаги выполняются внутри метода `applyBlur()`.

Заполните новую переменную с именем `blurBuilder`, создав `OneTimeWorkRequest` для рабочего размывания и вызвав функцию расширения `OneTimeWorkRequestBuilder` из `WorkManager KTX`.

data/WorkManagerBluromaticRepository.kt

```
import com.example.bluromatic.workers.BlurWorker
import androidx.work.OneTimeWorkRequestBuilder
...
override fun applyBlur(blurLevel: Int) {
    // Создаем рабочий запрос на размытие изображения
    val blurBuilder = OneTimeWorkRequestBuilder<BlurWorker>()
}
```

Запустите работу, вызвав метод `enqueue()` на вашем объекте `WorkManager`.

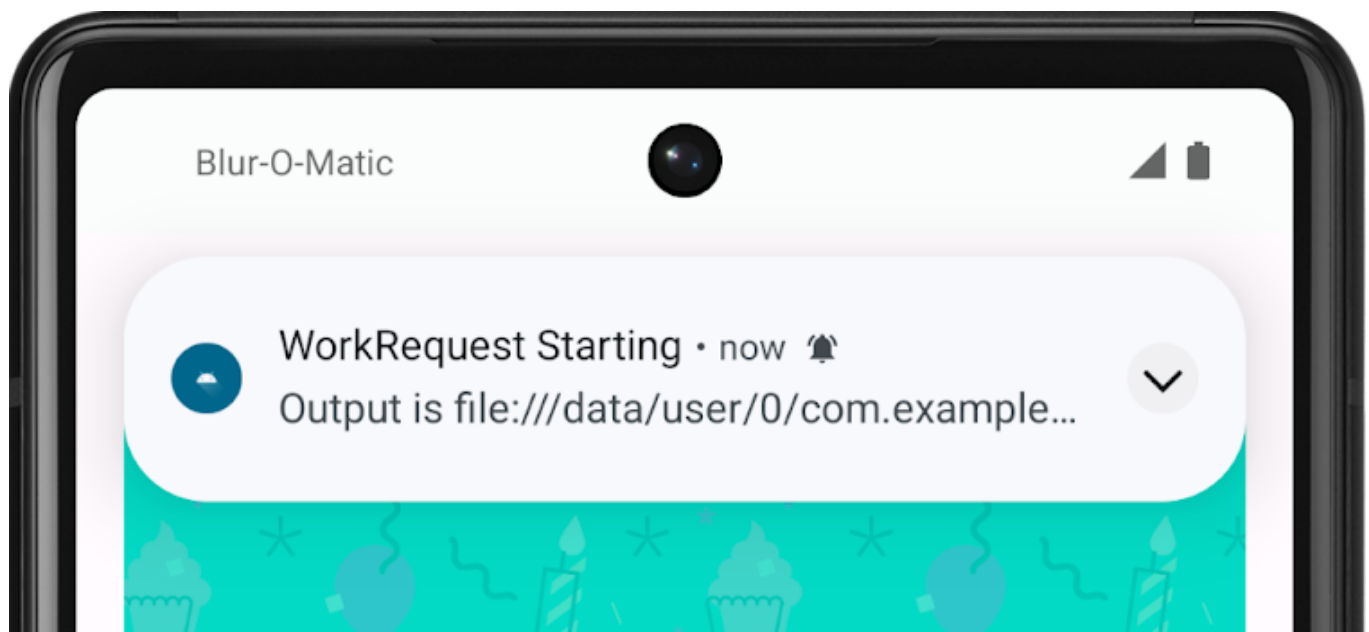
data/WorkManagerBluromaticRepository.kt

```
import com.example.bluromatic.workers.BlurWorker
import androidx.work.OneTimeWorkRequestBuilder
...
override fun applyBlur(blurLevel: Int) {
    // Создаем рабочий запрос на размытие изображения
    val blurBuilder = OneTimeWorkRequestBuilder<BlurWorker>()

    // Запуск работы
    workManager.enqueue(blurBuilder.build())
}
```

Запустите приложение и увидите уведомление при нажатии на кнопку Start. В это время изображение размывается одинаково, независимо от того, какой вариант вы выбрали. На последующих этапах степень размытия будет меняться в зависимости от выбранной опции.

Примечание: Приложение требует, чтобы уведомления были включены. Если уведомления не отображаются, перейдите в Настройки > Приложения > Blur-O-Matic > Уведомления и включите все уведомления Blur-O-Matic.





Select Blur Amount

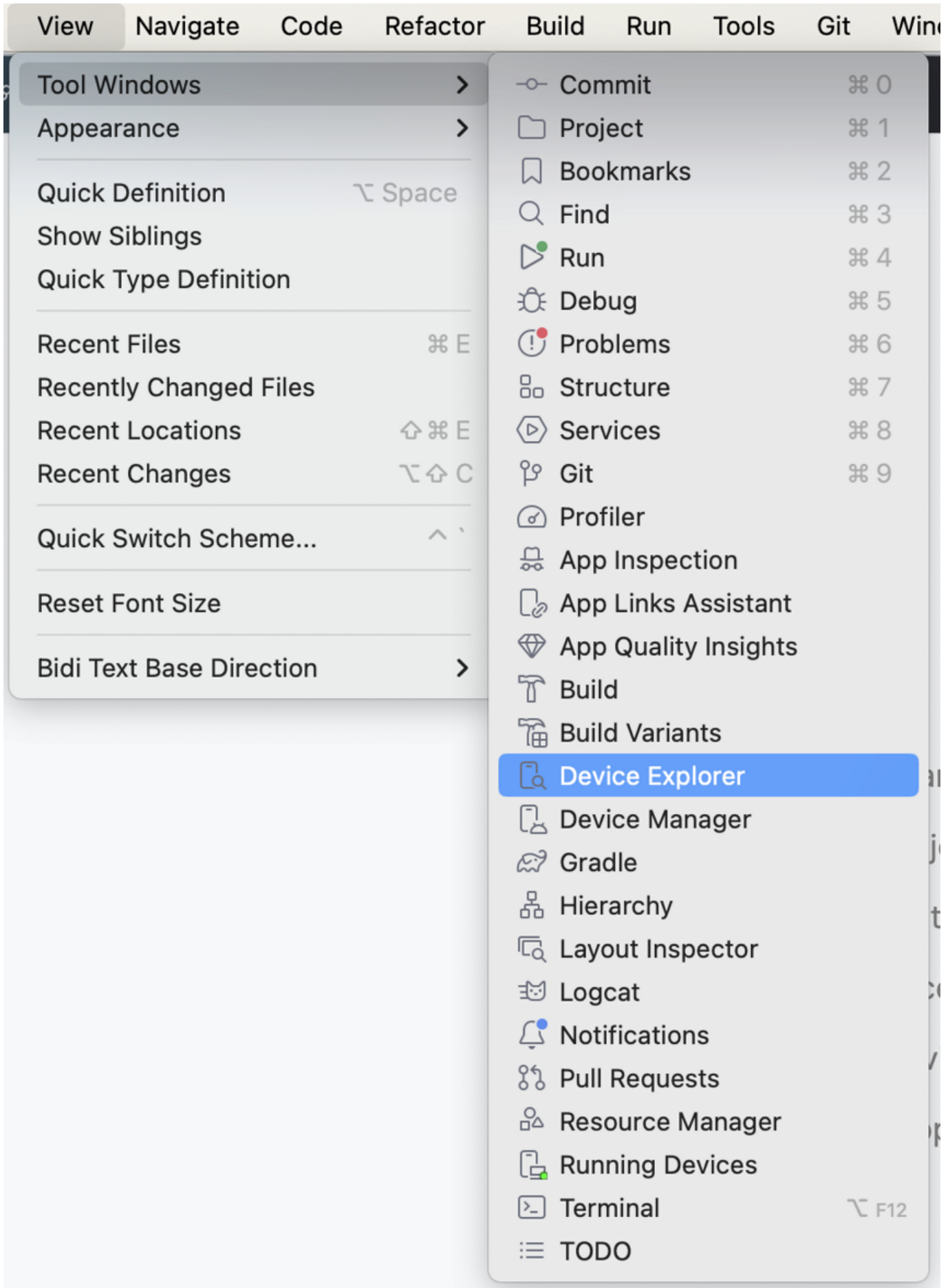
- ☒ A little blurred
- ☐ More blurred
- ☐ The most blurred

Start



{style=«width:400px»}

Чтобы убедиться, что изображение успешно размывается, можно открыть проводник устройства в Android Studio:



{style=«width:400px»}

Затем перейдите к `data > data > com.example.bluromatic > files > blur_filter_outputs >` и подтвердите, что изображение кекса действительно размыто:

Name	Permissions	Date	Size
> com.android.wallpaper.livepicker	drwx-----	2024-04-12 13:09	4 KB
> com.android.wallpaperbackup	drwx-----	2024-04-12 13:09	4 KB
> com.android.wallpapercropper	drwx-----	2024-04-12 13:09	4 KB
▼ com.example.bluromatic	drwx-----	2024-06-03 18:00	4 KB
> cache	drwxrws--x	2024-06-03 18:00	4 KB
> code_cache	drwxrws--x	2024-06-04 08:51	4 KB
> databases	drwxrwx--x	2024-06-03 18:00	4 KB
▼ files	drwxrwx--x	2024-06-04 08:53	4 KB
▼ blur_filter_outputs	drwx-----	2024-06-04 08:56	4 KB
 blur-filter-output-aaf7a378-a7a4-4859-be68-920013	-rw-----	2024-06-04 09:19	530.8 KB
≡ profileInstalled	-rw-----	2024-06-04 08:53	24 B
> no_backup	drwxrwx--x	2024-06-03 18:00	4 KB
> com.example.busschedule	drwx-----	2024-05-07 14:28	4 KB
> com.example.cupcake	drwx-----	2024-04-26 15:29	4 KB
> com.example.dessertclicker	drwx-----	2024-04-15 14:01	4 KB
> com.example.dessertrelease	drwx-----	2024-05-07 17:16	4 KB
> com.example.diceroller	drwx-----	2024-04-12 16:42	4 KB

{style=«width:400px»}

10. Входные данные и выходные данные

Размытие изображения в каталоге ресурсов - это хорошо, но чтобы Blur-O-Matic действительно стал революционным приложением для редактирования изображений, вам нужно позволить пользователю размыть изображение, которое он видит на экране, а затем показать ему результат размытия.

Для этого мы предоставляем URI отображаемого изображения кекса в качестве входа в наш WorkRequest, а затем используем выход нашего WorkRequest для отображения окончательного размытого изображения.

Input: URI of cupcake image



Blur image



Output: URI of temp blurred image

Входные и выходные данные передаются в рабочий и из него через объекты Data. Объекты Data - это легкие контейнеры для пар ключ/значение. Они предназначены для хранения небольшого количества данных, которые могут поступать в рабочий и выводиться из запроса на выполнение работы.

На следующем этапе вы передадите URI в BlurWorker, создав объект входных данных.

Создание объекта входных данных В файле `data/WorkManagerBluromaticRepository.kt`, внутри класса `WorkManagerBluromaticRepository`, создайте новую приватную переменную `imageUri`. Заполните переменную URI изображения, вызвав контекстный метод `getImageUri()`.

`data/WorkManagerBluromaticRepository.kt`

```
import com.example.blumatic.getImageUri
...
class WorkManagerBluromaticRepository(context: Context) : BluromaticRepository {

    private var imageUri: Uri = context.getImageUri() // <- Добавьте это
    private val workManager = WorkManager.getInstance(context)
    ...
}
```

Код приложения содержит вспомогательную функцию `createInputDataForWorkRequest()` для создания объектов входных данных.

data/WorkManagerBluromaticRepository.kt

```
// Для справки - уже существует в приложении
private fun createInputDataForWorkRequest(blurLevel: Int, imageUri: Uri): Data {
    val builder = Data.Builder()
    builder.putString(KEY_IMAGE_URI, imageUri.toString()).putInt(BLUR_LEVEL,
blurLevel)
    return builder.build()
}
```

Сначала вспомогательная функция создает объект `Data.Builder`. Затем она помещает в него `imageUri` и `blurLevel` в качестве пар ключ/значение. Затем создается объект `Data`, который возвращается при вызове функции `return builder.build()`.

Чтобы установить объект входных данных для `WorkRequest`, вы вызываете метод `blurBuilder.setInputData()`. Вы можете создать и передать объект данных за один шаг, вызвав в качестве аргумента вспомогательную функцию `createInputDataForWorkRequest()`. Для вызова функции `createInputDataForWorkRequest()` передайте переменную `blurLevel` и переменную `imageUri`.

data/WorkManagerBluromaticRepository.kt

```
override fun applyBlur(blurLevel: Int) {
    // Создаем рабочий запрос на размытие изображения
    val blurBuilder = OneTimeWorkRequestBuilder<BlurWorker>()

    // Новый код для объекта входных данных
    blurBuilder.setInputData(createInputDataForWorkRequest(blurLevel, imageUri))

    workManager.enqueue(blurBuilder.build())
}
```

Доступ к объекту входных данных Теперь давайте обновим метод `doWork()` в классе `BlurWorker`, чтобы получить `URI` и уровень размытия, которые были переданы объектом входных данных. Если значение для параметра `BlurLevel` не было указано, то по умолчанию оно будет равно 1.

Внутри метода `doWork()`:

Создайте новую переменную с именем `resourceUri` и заполните ее, вызвав `inputData.getString()` и передав константу `KEY_IMAGE_URI`, которая использовалась в качестве ключа при создании объекта входных данных. `val resourceUri = inputData.getString(KEY_IMAGE_URI)`

Создайте новую переменную с именем `blurLevel`. Заполните переменную, вызвав `inputData.getInt()` и передав в нее константу `BLUR_LEVEL`, которая использовалась в качестве ключа при создании объекта входных данных. В случае если эта пара ключ/значение не была создана, укажите значение по умолчанию 1 (один).

workers/BlurWorker.kt

```
import com.example.bluromatic.KEY_BLUR_LEVEL
import com.example.bluromatic.KEY_IMAGE_URI
...
override fun doWork(): Result {

    // ДОБАВЛЯЕМ ЭТИ СТРОКИ
    val resourceUri = inputData.getString(KEY_IMAGE_URI)
    val BlurLevel = inputData.getInt(KEY_BLUR_LEVEL, 1)

    // ... остальное doWork()
}
```

Получив URI, давайте теперь размоем изображение кекса на экране.

Проверьте, заполнена ли переменная resourceUri. Если она не заполнена, ваш код должен выбросить исключение. В следующем коде используется оператор require(), который выбрасывает исключение IllegalArgumentException, если первый аргумент оценивается как false.

workers/BlurWorker.kt

```
return@withContext try {
    // NEW code
    require(!resourceUri.isNullOrEmpty()) {
        val errorMessage =
            applicationContext.resources.getString(R.string.invalid_input_uri)
        Log.e(TAG, errorMessage)
        errorMessage
    }
}
```

Поскольку источник изображения передается в виде URI, нам нужен объект ContentResolver для чтения содержимого, на которое указывает URI.

Добавьте объект contentResolver к значению applicationContext.

workers/BlurWorker.kt

```
...
    require(!resourceUri.isNullOrEmpty()) {
        // ...
    }
    val resolver = applicationContext.contentResolver
    ...
```

Поскольку источником изображения теперь является переданный URI, используйте BitmapFactory.decodeStream() вместо BitmapFactory.decodeResource() для создания объекта Bitmap.

workers/BlurWorker.kt

```
import android.net.Uri
...
//      val picture = BitmapFactory.decodeResource(
//          applicationContext.resources,
//          R.drawable.android_cupcake
//      )

val resolver = applicationContext.contentResolver

val picture = BitmapFactory.decodeStream(
    resolver.openInputStream(Uri.parse(resourceUri))
)
```

Передайте переменную blurLevel в вызове функции blurBitmap().

workers/BlurWorker.kt

```
//val output = blurBitmap(picture, 1)
val output = blurBitmap(picture, blurLevel)
```

Создайте объект выходных данных Теперь вы закончили работу с этим рабочим и можете вернуть выходной URI в виде объекта выходных данных в Result.success(). Предоставление выходного URI в виде объекта выходных данных делает его легкодоступным для других рабочих для дальнейших операций. Этот подход будет полезен в следующем разделе, когда вы создадите цепочку рабочих.

Для этого выполните следующие шаги:

Перед кодом Result.success() создайте новую переменную с именем outputData. Заполните эту переменную вызовом функции workDataOf(), используя константу KEY_IMAGE_URI в качестве ключа и переменную outputUri в качестве значения. Функция workDataOf() создает объект Data из переданной пары ключа и значения.

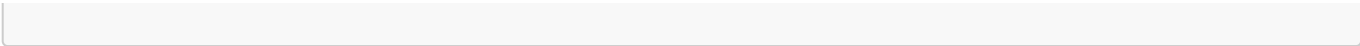
workers/BlurWorker.kt

```
import androidx.work.workDataOf
// ...
val outputData = workDataOf(KEY_IMAGE_URI to outputUri.toString())
```

Обновите код Result.success(), чтобы принять этот новый объект Data в качестве аргумента.

workers/BlurWorker.kt

```
//Result.success()
Result.success(outputData)
```



Удалите код, отображающий уведомление, поскольку он больше не нужен, так как объект выходных данных теперь использует URI.

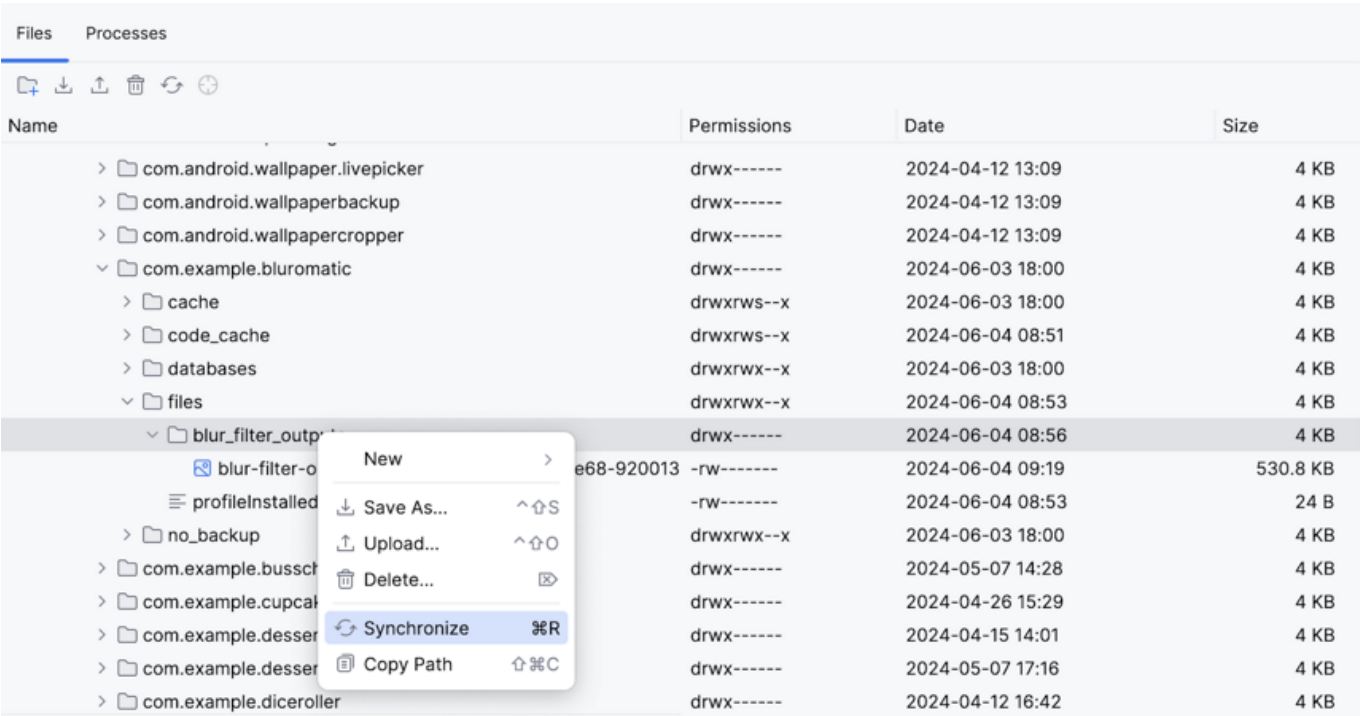
workers/BlurWorker.kt

```
//УДАЛИТЕ следующий код уведомления.  
//makeStatusNotification(  
// «Output is $outputUri»,  
// applicationContext  
//)  

```

Запустите ваше приложение На данном этапе, когда вы запускаете свое приложение, вы можете ожидать, что оно скомпилируется. Вы можете увидеть размытое изображение через проводник устройства, но пока не на экране.

Обратите внимание, что для просмотра изображений может потребоваться синхронизация:



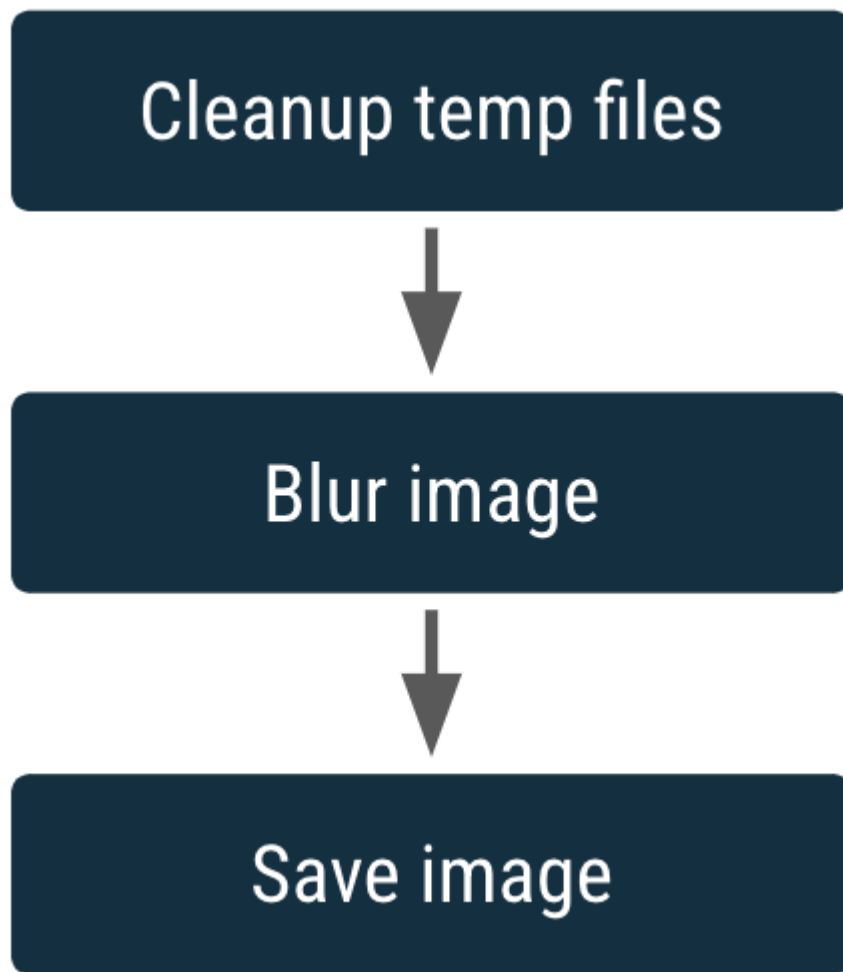
Отличная работа! Вы размыли входное изображение с помощью WorkManager!

11. Цепочка работ

Сейчас вы выполняете единственную задачу - размываете изображение. Эта задача - отличный первый шаг, но приложению все еще не хватает некоторых основных функций:

Приложение не очищает временные файлы. Приложение не сохраняет изображение в постоянный файл. Приложение всегда размывает изображение на одну и ту же величину. Чтобы добавить эту функциональность, можно использовать цепочку работ WorkManager. WorkManager позволяет создавать отдельные рабочие запросы, которые выполняются по порядку или параллельно.

В этом разделе вы создадите цепочку работ, которая будет выглядеть следующим образом:



{style=«width:400px»}

Коробки представляют собой рабочие запросы.

Еще одной особенностью цепочки является ее способность принимать входные данные и создавать выходные. Выход одного `WorkRequest` становится входом следующего `WorkRequest` в цепочке.

У вас уже есть `CoroutineWorker` для размытия изображения, но вам также нужен `CoroutineWorker` для очистки временных файлов и `CoroutineWorker` для постоянного сохранения изображения.

Создание `CleanupWorker` `CleanupWorker` удаляет временные файлы, если они существуют.

Щелкните правой кнопкой мыши на пакете `com.example.bluromatic.workers` в панели проекта Android и выберите `New -> Kotlin Class/File`. Назовите новый Kotlin-класс `CleanupWorker`. Скопируйте код `CleanupWorker.kt`, как показано в следующем примере кода. Поскольку работа с файлами не входит в задачи этого коделаба, вы можете скопировать следующий код для `CleanupWorker`.

`workers/CleanupWorker.kt`

```
package com.example.bluromatic.workers

import android.content.Context
import android.util.Log
```

```
import androidx.work.CoroutineWorker
import androidx.work.WorkerParameters
import com.example.bluromatic.DELAY_TIME_MILLIS
import com.example.bluromatic.OUTPUT_PATH
import com.example.bluromatic.R
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.delay
import kotlinx.coroutines.withContext
import java.io.File

/**
 * Cleans up temporary files generated during blurring process
 */
private const val TAG = "CleanupWorker"

class CleanupWorker(ctx: Context, params: WorkerParameters) : CoroutineWorker(ctx,
params) {

    override suspend fun doWork(): Result {
        /** Makes a notification when the work starts and slows down the work so
        that it's easier
        * to see each WorkRequest start, even on emulated devices
        */
        makeStatusNotification(
            applicationContext.resources.getString(R.string.cleaning_up_files),
            applicationContext
        )

        return withContext(Dispatchers.IO) {
            delay(DELAY_TIME_MILLIS)

            return@withContext try {
                val outputDirectory = File(applicationContext.filesDir,
OUTPUT_PATH)
                if (outputDirectory.exists()) {
                    val entries = outputDirectory.listFiles()
                    if (entries != null) {
                        for (entry in entries) {
                            val name = entry.name
                            if (name.isNotEmpty() && name.endsWith(".png")) {
                                val deleted = entry.delete()
                                Log.i(TAG, "Deleted $name - $deleted")
                            }
                        }
                    }
                }
                Result.success()
            } catch (exception: Exception) {
                Log.e(
                    TAG,

applicationContext.resources.getString(R.string.error_cleaning_file),
                    exception
                )
            }
        }
    }
}
```

```

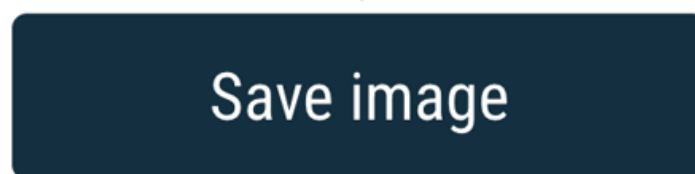
        }
    }
}

Result.failure()
    
```

Создание `SaveImageToFileWorker` Класс `SaveImageToFileWorker` сохраняет временный файл в постоянный файл.

`SaveImageToFileWorker` принимает входные и выходные данные. На входе - строка URI временно размытого изображения, хранящаяся с ключом `KEY_IMAGE_URI`. На выходе - строка URI сохраненного размытого изображения, хранящаяся с ключом `KEY_IMAGE_URI`.

Input: URI of temp blurred image



Output: URI of saved final blurred image

{style=«width:400px»}

Щелкните правой кнопкой мыши на пакете `com.example.bluromatic.workers` в панели проекта Android и выберите `New -> Kotlin Class/File`. Назовите новый Kotlin-класс `SaveImageToFileWorker`. Скопируйте код `SaveImageToFileWorker.kt`, как показано в следующем примере. Поскольку работа с файлами не входит в задачи данного коделаба, вы можете скопировать следующий код для `SaveImageToFileWorker`. В представленном коде обратите внимание на то, как значения `resourceUri` и `output` извлекаются и хранятся с ключом `KEY_IMAGE_URI`. Этот процесс очень похож на код, который вы ранее написали для объектов входных и выходных данных.

`workers/SaveImageToFileWorker.kt`

```

package com.example.bluromatic.workers

import android.content.Context
import android.graphics.BitmapFactory
import android.net.Uri
import android.provider.MediaStore
import android.util.Log
import androidx.work.CoroutineWorker
import androidx.work.WorkerParameters
import androidx.work.workDataOf
import com.example.bluromatic.DELAY_TIME_MILLIS
import com.example.bluromatic.KEY_IMAGE_URI
import com.example.bluromatic.R
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.delay
import kotlinx.coroutines.withContext
import java.text.SimpleDateFormat
import java.util.Locale
import java.util.Date

/**
 * Saves the image to a permanent file
 */
private const val TAG = "SaveImageToFileWorker"

class SaveImageToFileWorker(ctx: Context, params: WorkerParameters) :
    CoroutineWorker(ctx, params) {

    private val title = "Blurred Image"
    private val dateFormatter = SimpleDateFormat(
        "yyyy.MM.dd 'at' HH:mm:ss z",
        Locale.getDefault()
    )

    override suspend fun doWork(): Result {
        // Makes a notification when the work starts and slows down the work so
        that
        // it's easier to see each WorkRequest start, even on emulated devices
        makeStatusNotification(
            applicationContext.resources.getString(R.string.saving_image),
            applicationContext
        )

        return withContext(Dispatchers.IO) {
            delay(DELAY_TIME_MILLIS)

            val resolver = applicationContext.contentResolver
            return@withContext try {
                val resourceUri = inputData.getString(KEY_IMAGE_URI)
                val bitmap = BitmapFactory.decodeStream(
                    resolver.openInputStream(Uri.parse(resourceUri))
                )
            }
        }
    }
}

```

```

        val imageUrl = MediaStore.Images.Media.insertImage(
            resolver, bitmap, title, dateFormatter.format(Date())
        )
        if (!imageUrl.isNullOrEmpty()) {
            val output = workDataOf(KEY_IMAGE_URI to imageUrl)

            Result.success(output)
        } else {
            Log.e(
                TAG,

applicationContext.resources.getString(R.string.writing_to_mediaStore_failed)
            )
            Result.failure()
        }
    } catch (exception: Exception) {
        Log.e(
            TAG,

applicationContext.resources.getString(R.string.error_saving_image),
            exception
        )
        Result.failure()
    }
}
}
}
}
}

```

Примечание: В предоставленном коде для рабочего CleanupWorker и рабочего SaveImageToFileWorker содержится оператор `delay(DELAY_TIME_MILLIS)`. Этот код замедляет работу рабочего во время его выполнения. Этот код был включен в учебные цели, чтобы вам было легче увидеть работающих рабочих в инспекторе фоновых задач, а также для обеспечения короткой паузы между сообщениями уведомлений. Обычно этот код не используется в производственном коде.

Создание цепочки работ В настоящее время код создает и запускает только один `WorkRequest`.

На этом шаге вы измените код так, чтобы он создавал и выполнял цепочку рабочих запросов вместо одного запроса на изображение размытия.

В цепочке `WorkRequests` первым рабочим запросом будет очистка временных файлов.

Вместо вызова `OneTimeWorkRequestBuilder` вызовите `workManager.beginWith()`. Вызов метода `beginWith()` возвращает объект `WorkContinuation` и создает начальную точку для цепочки `WorkRequests` с первым рабочим запросом в цепочке.

`data/WorkManagerBluromaticRepository.kt`

```

import androidx.work.OneTimeWorkRequest
import com.example.bluromatic.workers.CleanupWorker
// ...

```

```

    override fun applyBlur(blurLevel: Int) {
        // Add WorkRequest to Cleanup temporary images
        var continuation =
            workManager.beginWith(OneTimeWorkRequest.from(CleanupWorker::class.java))

        // Add WorkRequest to blur the image
        val blurBuilder = OneTimeWorkRequestBuilder<BlurWorker>()
        ...
    }

```

Примечание: В этом коде мы показываем альтернативный способ создания объекта `OneTimeWorkRequest`. Вызов `OneTimeWorkRequest.from(CleanupWorker::class.java)` эквивалентен вызову `OneTimeWorkRequestBuilder().build()`. Класс `OneTimeWorkRequest` берется из библиотеки `AndroidX Work`, а `OneTimeWorkRequestBuilder` - это вспомогательная функция, предоставляемая расширением `WorkManager KTX`.

Вы можете дополнить эту цепочку запросов на выполнение работы, вызвав метод `then()` и передав ему объект `WorkRequest`.

Удалите вызов `workManager.enqueue(blurBuilder.build())`, который выполнял только один рабочий запрос. Добавьте следующий рабочий запрос в цепочку, вызвав метод `.then()`.

`data/WorkManagerBluromaticRepository.kt`

```

...
//workManager.enqueue(blurBuilder.build())

// Добавляем запрос на работу с размытием в цепочку
continuation = continuation.then(blurBuilder.build())
...

```

Создайте рабочий запрос для сохранения изображения и добавьте его в цепочку.

`data/WorkManagerBluromaticRepository.kt`

```

import com.example.bluromatic.workers.SaveImageToFileWorker

...
continuation = continuation.then(blurBuilder.build())

// Добавляем WorkRequest для сохранения изображения в файловой системе
val save = OneTimeWorkRequestBuilder<SaveImageToFileWorker>()
    .build()
continuation = continuation.then(save)
...

```

Чтобы начать работу, вызовите метод `enqueue()` на объекте `continuation`.

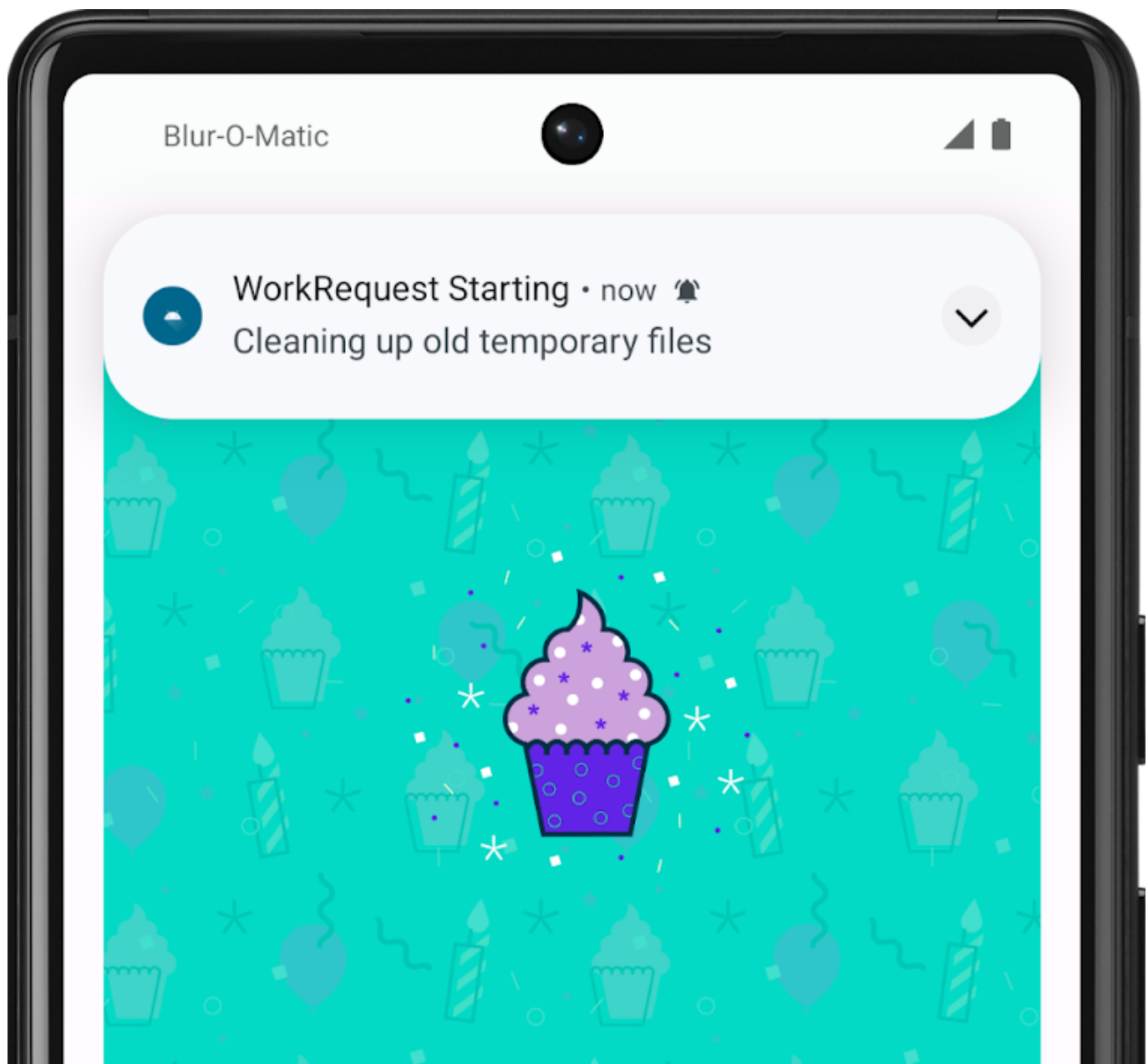
`data/WorkManagerBluromaticRepository.kt`

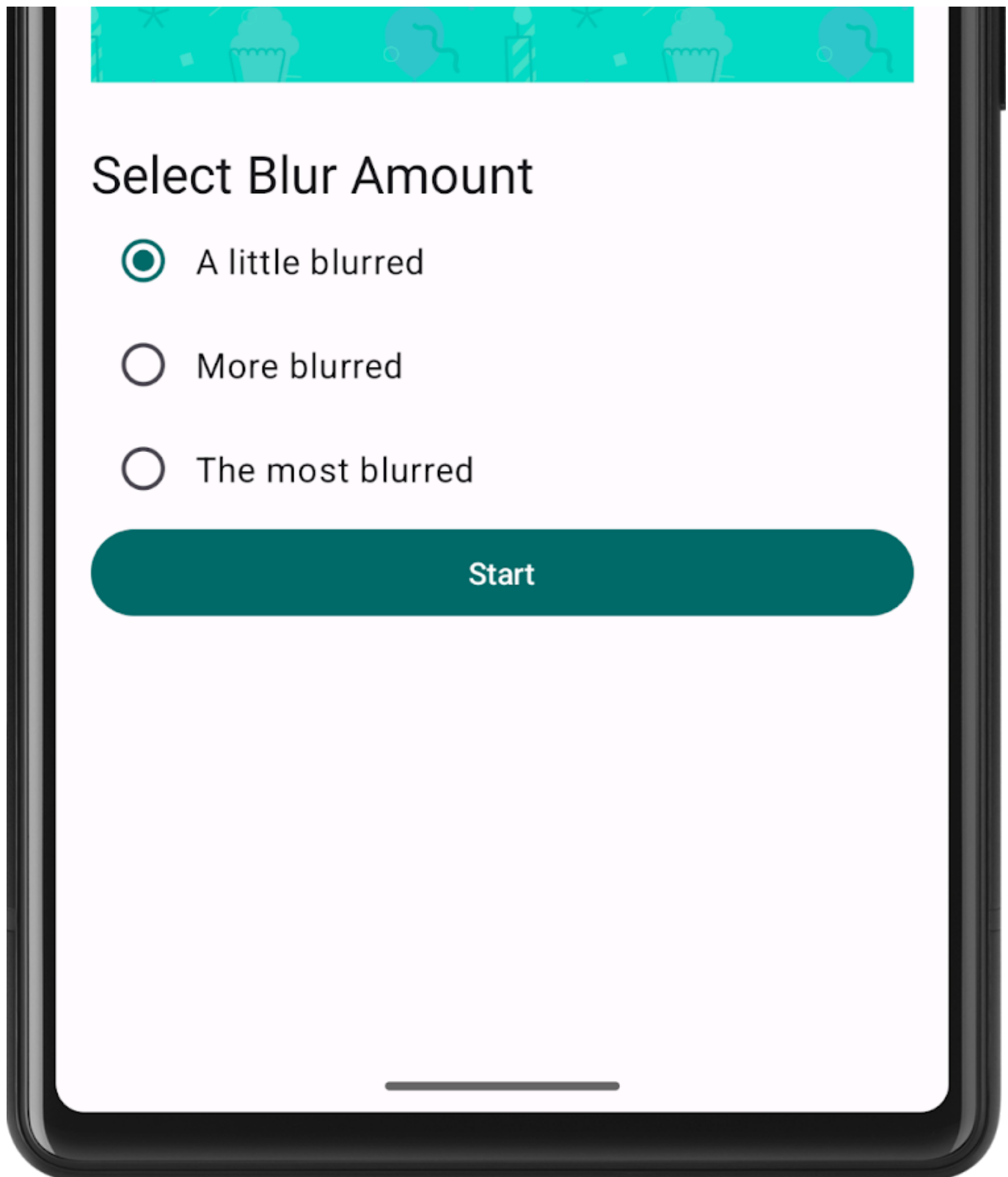
```
...  
continuation = continuation.then(save)  
  
// Запуск работы  
continuation.enqueue()  
...
```

Этот код создает и запускает следующую цепочку рабочих запросов: рабочий запрос CleanupWorker, затем рабочий запрос BlurWorker, затем рабочий запрос SaveImageToFileWorker.

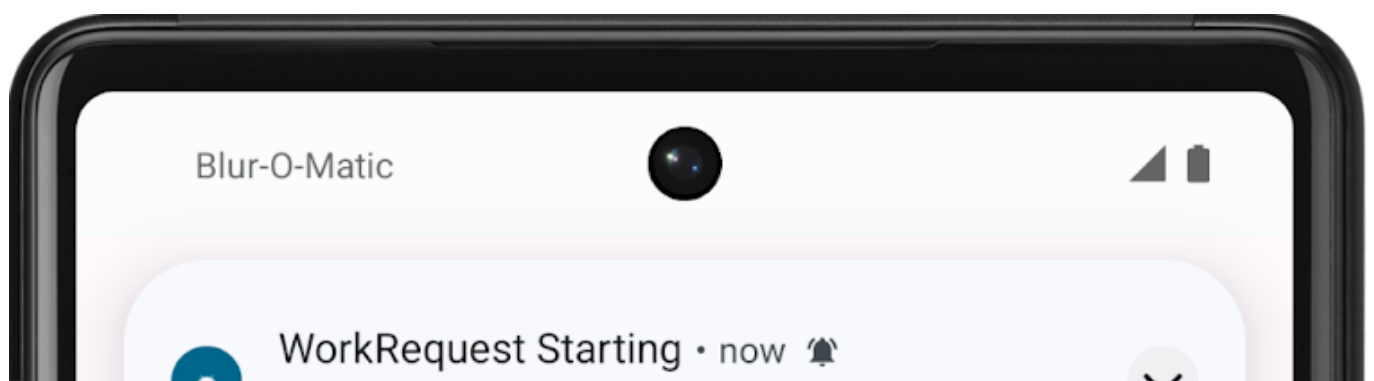
Запустите приложение. Теперь вы можете нажать кнопку Start и увидеть уведомления о выполнении различных рабочих операций. Вы все еще можете видеть размытое изображение в проводнике устройства, и в следующем разделе вы добавите дополнительную кнопку, чтобы пользователи могли видеть размытое изображение на устройстве.

На следующих скриншотах можно заметить, что в уведомлении отображается, какой рабочий запущен в данный момент.





{style="width:400px"}



Blurring image



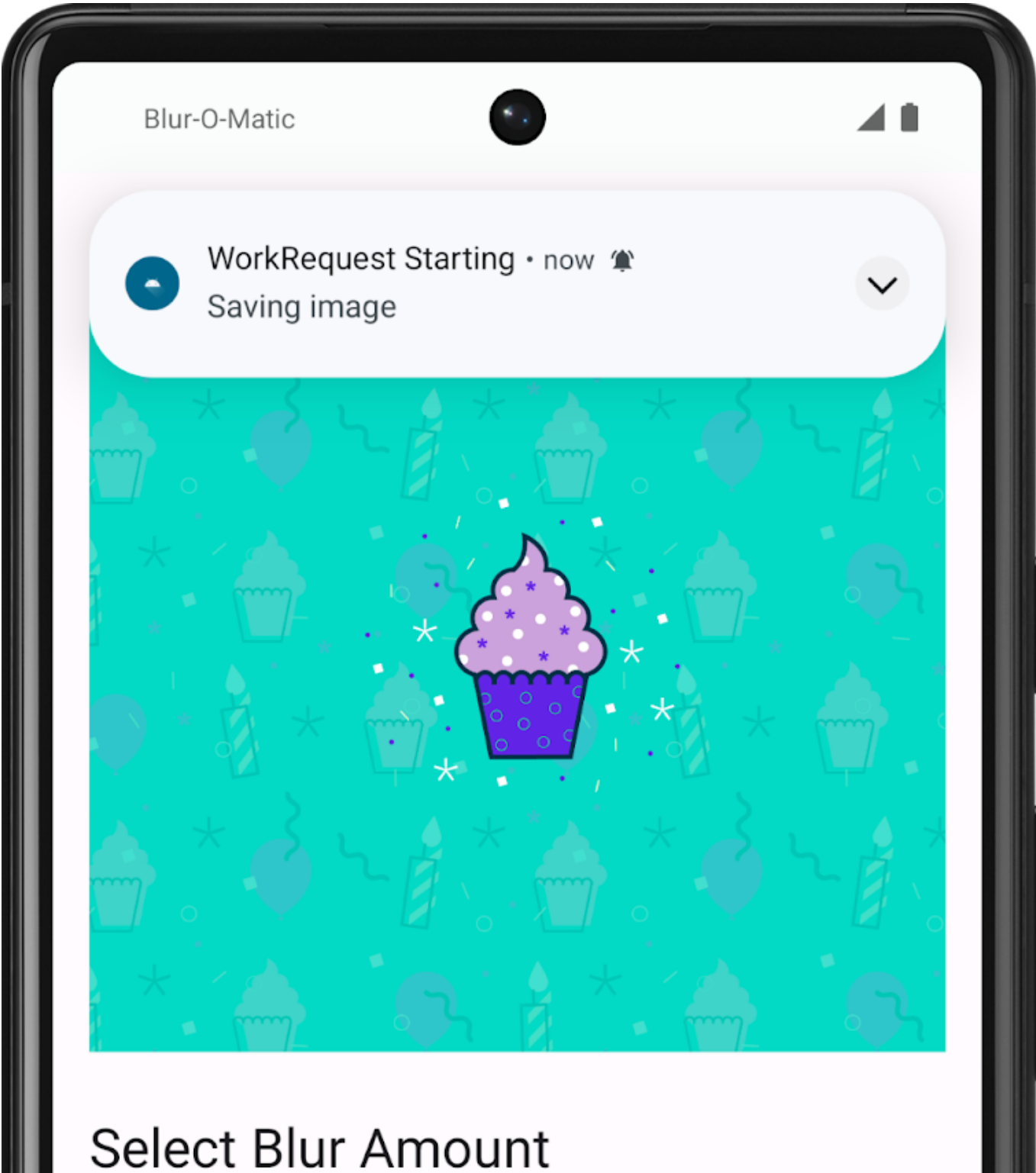
Select Blur Amount

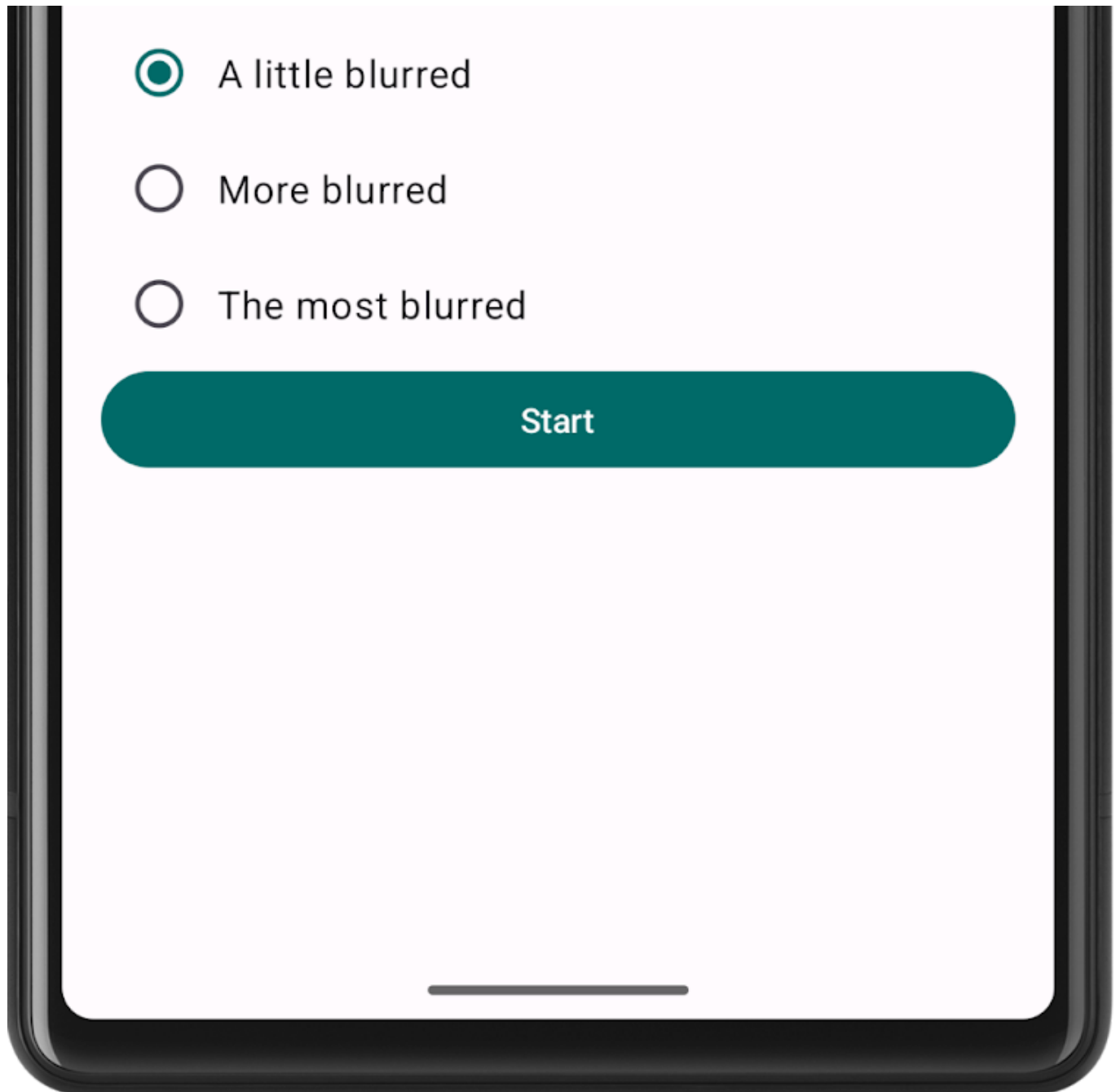
- ☒ A little blurred
- ☐ More blurred
- ☐ The most blurred

Start



{style="width:400px"}





{style="width:400px"}

Обратите внимание, что в выходной папке содержится несколько размытых изображений - изображения, находящиеся на промежуточных стадиях размытия, и финальное изображение, на котором отображено изображение с выбранной вами степенью размытия.

Великолепная работа! Теперь вы можете очистить временные файлы, размыть изображение и сохранить его!

Получите код решения

Чтобы загрузить код готового коделаба, вы можете воспользоваться следующими командами:

```
$ git clone https://github.com/google-developer-training/basic-android-kotlin-compose-training-workmanager.git
$ cd basic-android-kotlin-compose-training-workmanager
$ git checkout intermediate
```

Заключение

Вы закончили работу над приложением Blur-O-Matic и в процессе работы узнали о:

- Добавление WorkManager в ваш проект
- Планирование одноразового рабочего запроса
- Входные и выходные параметры
- Объединение рабочих запросов в цепочку
- WorkManager поддерживает гораздо больше, чем мы смогли рассмотреть в этом коделабе, включая повторяющуюся работу, библиотеку поддержки тестирования, параллельные рабочие запросы и объединение входов.